

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 03: Control de Flujo - Condicionales

«El Guardia de la Puerta y el Menú de Opciones»

Toma de decisiones lógicas en Python: operadores relacionales, operadores lógicos booleanos
(and, or, not) y estructuras if, elif y else.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante Absoluto | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Bifurcaciones Lógicas y Toma de Decisiones	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Árbol de Decisión y Evaluación de Condiciones	Pág. 5
Capítulo 3	Implementación Práctica en Python: Sistema de Clasificación de Préstamos Bancarios	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Errores Frecuentes con Condicionales	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la evaluación de expresiones booleanas y la exclusión mutua en cadenas if-elif-else.



Competencia Práctica

Implementar sistemas de validación de reglas de negocio, control de acceso y árboles de decisión.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Bifurcaciones Lógicas y Toma de Decisiones

Un programa no es una línea recta; es un camino con encrucijadas donde el flujo toma una dirección según las condiciones.

☀ Metáfora Central: El Guardia de la Puerta y el Menú de Opciones

Imagina un guardia en la entrada de un club: revisa tu entrada (if). Si tienes pase VIP entra gratis (if), si tienes entrada general paga boleto (elif), y si no tienes entrada se le deniega el acceso (else).

Principios Teóricos y Modelo Mental

Operadores relacionales: == (igualdad), != (diferente), > (mayor), < (menor), >= (mayor o igual), <= (menor o igual).

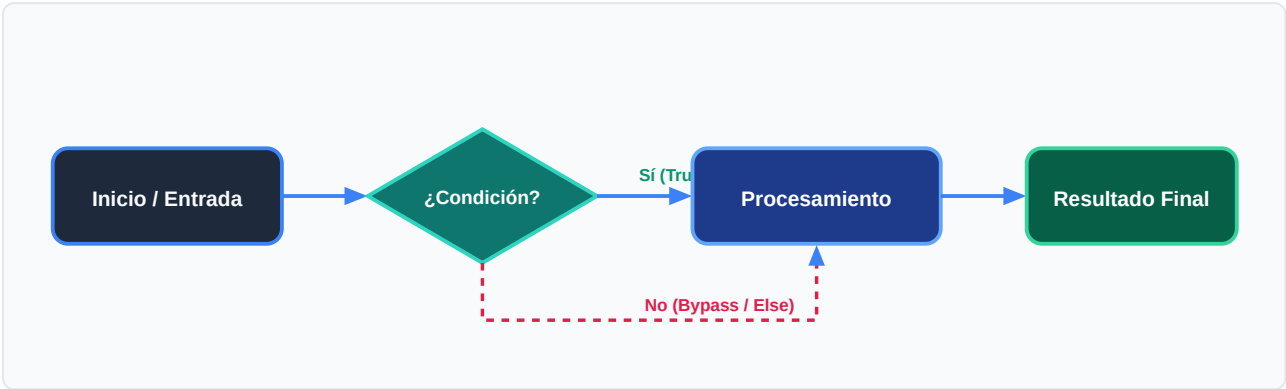
Operadores lógicos: and (ambas condiciones deben ser True), or (al menos una True), not (invierte el valor de verdad).

⚡ Regla de Oro en Python

En una cadena if-elif-else, tan pronto como una condición resulta True, se ejecuta su bloque y se omiten todas las demás.

2. Árbol de Decisión y Evaluación de Condiciones

Representación del flujo booleano con múltiples alternativas excluyentes.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Evalúa la primera condición del if principal.	Condición 1: ¿edad >= 18?
2. Evaluación	Si es True, entra al bloque if y salta al final de la estructura.	Ejecuta bloque prioritario
3. Transformación	Si es False, evalúa secuencialmente los bloques elif.	Condición 2: ¿tiene_permiso?
4. Retorno / Salida	Si ninguna condición previa fue True, se ejecuta el bloque else por defecto.	Rama fallback de seguridad



Visualización Mental

Ordena tus condiciones de la más específica a la más general para evitar que un caso amplio oculte casos particulares.

3. Sistema de Clasificación de Préstamos Bancarios

Ejemplo práctico con operadores lógicos combinados y evaluación de reglas financieras:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
salario = float(input("Salario mensual ($): "))
puntaje_credito = int(input("Puntaje crediticio (300-850): "))
tiene_deudas = input("¿Tiene deudas activas? (s/n): ").lower() == "s"

if salario >= 3000.0 and puntaje_credito >= 720 and not tiene_deudas:
    estado = "Aprobado Premium (Tasa de interés preferencial)"
elif salario >= 1800.0 and puntaje_credito >= 650:
    estado = "Aprobado Estándar (Sujeto a verificación)"
elif salario >= 1200.0 or puntaje_credito >= 600:
    estado = "Requiere Codeudor o Aval"
else:
    estado = "Rechazado (No cumple los requisitos mínimos)"

print(f"\nResultado de la solicitud: {estado}")
```

Análisis del Código Fuente

El código implementa lógica booleana compuesta con and, not y or, garantizando una jerarquía de evaluación limpia.

4. Errores Frecuentes con Condicionales

Trampas clásicas de sintaxis y lógica booleana en Python:

⚠ Gotcha Frecuente (Trampa de Principiante)

Confundir el operador de asignación (=) con el operador de comparación (==).

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
if rol = "admin": # SyntaxError
    print("Acceso total")
```

✓ Patrón Pythonic / Correcto

```
if rol == "admin": # Comparación correcta
    print("Acceso total")
```

🛡 Consejo de Resiliencia en Producción

Aprovecha la evaluación de cortocircuito (short-circuit evaluation) en Python para proteger llamadas riesgosas.

5. Resumen Ejecutivo y Conclusiones

Has dominado el núcleo de la toma de decisiones en software mediante condicionales y lógica booleana.



Logro Alcanzado en esta Clase

Capacidad para codificar flujos lógicos complejos y reglas de negocio robustas.

Notas del Instructor y Siguientes Pasos

En la próxima clase abordaremos la repetición inteligente: bucles for y while para procesar volúmenes masivos de datos.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Diseña un sistema de tarificación de boletos de cine con descuentos por edad, día de la semana y membresía VIP.